

Sistemas de Arquivos Distribuídos (DFS)

- Emula a interface de um sistema de arquivos local, mas com dados em servidores remotos
- Ideia central: **transparência** (esconder a rede do usuário e do programador)
- Requisito importante: lidar com **atualizações concorrentes**
 - Conceito: *one-copy semantics* (consistência de cópia única)

Transparência

- Acesso
 - Mesmas operações para arquivo local e remoto (ex: open, read, write)
 - Permite aplicações legadas funcionarem sem alteração
- Localização
 - Espaço de nomes uniforme (caminho não revela IP/local físico)
 - Permite migrar arquivos no servidor sem quebrar links
- Mobilidade
 - Arquivo pode mudar de servidor/disco enquanto está em uso
- Falha
 - Oculta erros de hardware/software
 - Sistema tenta reconectar/usar réplicas automaticamente

Escalabilidade

- Suportar crescimento de usuários/recursos sem degradar proporcionalmente o desempenho
- Ex clássico: **AFS (Andrew File System)**, pensado para ambientes com milhares de estações

Papel do Middleware no DFS

- DFS atua como camada entre o SO local e as aplicações
- Abstrai a heterogeneidade dos protocolos de rede subjacentes

Arquitetura: Serviço de Arquivos Plano + Serviço de Diretório

- **Serviço de Arquivos Plano**

- Opera no conteúdo: Read(FileId, i, n), Write(FileId, i, Data), Create()
- Suporta acesso aleatório por deslocamento i (importante p/ bases de dados)
- Usa **UFIDs** (Unique File Identifiers)
 - IDs globalmente únicos (diferente de i-node local)
 - Este nível não entende nomes textuais, só IDs
- **Serviço de Diretório**
 - Mapeia nomes (strings) → UFIDs
 - Diretórios como “arquivos especiais” com referências (UFIDs) para outros arquivos
 - Resolução iterativa de nomes: caminho completo exige várias chamadas (uma por componente)
 - Otimização no cliente: cache de prefixos
- **Módulo cliente**
 - Integra serviço plano + diretório e oferece a API final
 - Onde fica a maior parte da inteligência de cache

Projeto sem Estado (Stateless)

- Servidor não mantém quais arquivos estão abertos nem quais clientes conectados
- Vantagem: recuperação pós-crash é bem mais simples (não há estado para restaurar)

Sun Network File System (NFS)

- Lançado em 1984, objetivo: portabilidade e independência de hardware/SO
- Virou padrão de fato na indústria

NFS: VFS e Montagem

- Introduz **VFS (Virtual File System)** no kernel
 - Intercepta syscalls e decide se o alvo é local ou remoto
 - Fluxo: Aplicação → VFS → (FS local ou Cliente NFS)
- Usa **v-nodes** para identificar arquivos na rede

- Montagem: cliente monta diretórios remotos em subárvores locais (ex: /usr/students, /usr/staff)

NFS: Protocolo (RPC)

- Comunicação via **RPC**
 - Operações: lookup, read, write, getattr, setattr
- NFS v2/v3: design **stateless** para robustez
- Muitas operações são **idempotentes**
 - Importante para lidar com perda/duplicação de mensagens (especialmente sobre UDP/IP)

Cache e Consistência no NFS

- Cache no servidor (RAM)
 - Escrita em disco pode ser síncrona ou postergada
 - Commit síncrono = mais durável, porém mais lento
- Cache no cliente (dados + metadados)
 - Melhora desempenho, mas complica consistência
- Consistência por *polling* (timestamps)
 - Cliente compara timestamp local vs servidor periodicamente (3–30s)
 - Cria janela de inconsistência
- Atualização perdida
 - Dois clientes podem escrever no mesmo bloco; o último write sobrescreve o anterior
 - Protocolo básico não oferece locks fortes nativos
- Segurança
 - RPC autenticado, UID/GID UNIX
 - Pode usar Kerberos para evitar falsificação/interceptação
- WebNFS
 - Extensão para acesso via HTTP (ex: navegador acessando repositórios NFS)

NFS v4: mudança de paradigma

- Migra para modelo **stateful**
- Objetivo: melhor gerenciamento de cache e delegação (aproxima do modelo do AFS)

Andrew File System (AFS)

- Criado na CMU com foco em **escalabilidade massiva** (WAN/campus com >1000 estações)
- Meta: reduzir o gargalo do servidor central

Cache de arquivo inteiro (AFS)

- Ao abrir: cliente transfere o arquivo inteiro para o disco local
- Leituras/escritas subsequentes ocorrem localmente (alta performance)

Vice e Venus

- **Vice (servidor):** gerencia árvore global e metadados
- **Venus (cliente):** gerencia cache local e intercepta chamadas do kernel

Open e Callbacks

- No open, servidor retorna o arquivo + **callback promise**
- Servidor mantém estado de quem tem cópia
 - Se alguém modifica, servidor notifica os demais e invalida caches
 - Elimina necessidade de polling

Semântica de sessão

- Alterações ficam visíveis para outros **após o close**
- Se dois clientes editam ao mesmo tempo, a última versão fechada prevalece

Escalabilidade e administração

- Comunicação cliente-servidor concentrada em *open/close* → tráfego baixo
- **Volumes**
 - Agrupamento administrativo (ex: home de usuário)
 - Pode migrar volumes entre servidores transparentemente
- **Células**
 - Unidade administrativa independente (ex: cmu.edu)

- Permite federação de múltiplas instituições numa árvore global

Falhas no AFS

- Como é stateful (callbacks), recuperação é mais complexa
- Após reinício do servidor, clientes precisam revalidar callbacks

AFS-3 e arquivos grandes

- Evolui para transferência em **chunks** menores (p/ arquivos que não cabem inteiros na cache)

Google File System (GFS)

- Feito para necessidades massivas (índices web, logs, etc.)
- Premissa: priorizar **throughput** mais que baixa latência
- Falhas são “normais” em clusters enormes → tolerância a falhas via software

Chunks, chunkservers e Master

- Arquivos divididos em **chunks de 64 MB**
 - Reduz interação contínua com metadados
- **Master** único ativo
 - Gerencia nomes, permissões e localização de chunks
 - Fica fora do *data path*: cliente pega rota e fala direto com chunkservers

Replicação e leitura

- Cada chunk replicado (padrão: 3 cópias), normalmente em racks diferentes
- Leitura:
 1. Cliente pergunta ao Master onde está o chunk
 2. Master retorna endereços das réplicas
 3. Cliente lê do chunkserver mais próximo

Record-append e consistência

- **Record-append**: append atômico por múltiplos clientes no mesmo arquivo (útil p/ logs/MapReduce)
- Consistência relaxada

- Garantia “pelo menos uma vez” → pode haver duplicatas
- Aplicação lida com consistência final

Leases e Chubby

- Master concede **lease** a um chunkserver primário
 - Primário ordena escritas, evitando coordenação fina do Master
- **Chubby**: serviço de lock/coordenação
 - Eleição segura do Master e pequenos arquivos de configuração
 - Baseado em Paxos

Comparação final (NFS vs AFS vs GFS)

- NFS
 - Foco em LAN e uso geral (documentos/ambientes de dev)
 - Cache em RAM + consistência via polling (mais fraca)
- AFS
 - Foco em WAN/campus grande
 - Cache persistente de arquivo inteiro + callbacks + semântica de sessão
- GFS
 - Foco em data centers e Big Data
 - Chunks grandes, replicação e sem cache “convencional” (streaming/append)

Resumo (o que vimos hoje?)

- DFS e seus objetivos: transparência, escalabilidade e concorrência
- Arquitetura (serviço plano + diretório) e o papel do módulo cliente
- NFS (RPC, stateless, caching/polling, limitações de concorrência, evolução no v4)
- AFS (cache local, callbacks, semântica de sessão, volumes/células, recuperação de falhas)
- GFS (chunks 64MB, master fora do data path, replicação, record-append, consistência relaxada, leases/Chubby)